

NeuroEdge: Anomaly Detection for Edge Cybersecurity Using Neuromorphic Computing

Project Report

Submitted by

BL.EN.U4EAC22033

Lasyapriya Bharadwaj K.

BL.EN.U4EAC22047

Shashank R.

Submitted to

Mr. Sagar B.

Assistant Professor (Sr. Gr.)

in partial fulfillment for the award of the degree
of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMPUTER ENGINEERING



DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

AMRITA SCHOOL OF ENGINEERING
AMRITA VISHWA VIDYAPEETHAM
BENGALURU-560035

Abstract

The proliferation of Internet of Things (IoT) devices has created a vast attack surface, making robust network security essential. Intrusion Detection Systems (IDS) are critical for protecting these networks, yet they face significant challenges with real-time detection and severe class imbalance in security datasets. Existing studies often report high offline accuracy but neglect real-time deployment challenges and fail to address the poor handling of minority classes, such as the frequent misclassification of benign traffic. Furthermore, energy-efficient neuromorphic models remain under-explored for this domain. This project proposes a hybrid intrusion detection framework utilizing both classical classifiers and a novel Spiking Neural Network (SNN). Its novelty lies in the full-pipeline integration—from live packet capture with Argus and a concurrent Go-based processing pipeline to real-time inference on containerized edge devices. Crucially, we introduce an augmented and balanced dataset (DB5), developed using SMOTE and synthetic generation of normal traffic, to resolve class imbalance. The proposed SNN model, trained on the DB5 dataset, achieves robust detection accuracy exceeding 96%. This approach significantly improves minority class prediction and resolves the misclassification of normal traffic, demonstrating superior, reliable performance over models trained on the original imbalanced data. This work validates the potential of a neuromorphic-assisted framework for adaptive and scalable IoT security, offering a practical foundation for efficient intrusion detection on resource-constrained edge devices.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem Statement	1
1.3	Gaps in Existing Research	2
1.4	Objectives of the Project	2
1.5	Proposed Methodology and Novelty	2
1.6	Expected Results and Impact	3
2	Literature Survey	4
2.0.1	Summary	6
2.0.2	Key Takeaways	7
2.0.3	Open Challenges	7
3	System Design	8
3.1	Background	9
3.1.1	Edge Devices	9
3.1.2	Attacks on Edge Devices	9
3.2	Data Acquisition and Preperation	10
3.2.1	Bot-IoT Dataset Overview	11
3.2.2	Combined Dataset Creation	12
3.2.3	Feature Selection	13
3.3	Dataset Labeling and Classification Setup	15
3.3.1	Creation of DB2	16
3.3.2	Data Preprocessing	16

3.3.3	Subsequent Database Versions (DB3-DB5)	18
3.4	Model Training and Evaluation	20
3.4.1	Training and Evaluation Setup	20
3.4.2	Limitations and Iterations	22
3.5	Real-Time Prediction Pipeline	23
3.5.1	Feature Vector Generation	24
3.5.2	Prediction Service	27
3.5.3	Edge Device Deployment and Traffic Generation	27
3.6	Result Visualization Dashboard	28
3.6.1	Dashboard Architecture	28
3.6.2	User Interface Components	30
4	Results and Discussion	32
4.0.1	Model Performance Evaluation	32
4.0.2	Real-World Attack Evaluation and Prediction Quality	38
4.0.3	Comparative Analysis	39
4.0.4	Limitations and Future Work	40
5	Conclusion	41
	References	42

List of Figures

3.1	System workflow of the Intrusion Detection Framework.	8
3.2	Block diagram of Data Acquisition and Preparation.	11
3.3	Block diagram of Combined Dataset Creation.	12
3.4	Data Preprocessing Steps.	17
3.5	Flowchart of Model Training and Evaluation.	20
3.6	The Leaky-Integrate and Fire (LIF) model.	21
3.7	Flowchart of Creation of DB3, DB4 and DB5.	23
3.8	Block diagram of the Real-Time Prediction Pipeline.	24
3.9	Message pushed into the flow_aggregator Redis channel.	26
3.10	Message pushed into the feature_vectors Redis channel.	26
3.11	Message pushed into the prediction_results Redis channel.	27
3.12	Architecture of the Result Visualization Dashboard.	29
3.13	Mockup of the Dashboard with no Attacks.	29
3.14	Mockup of the Dashboard with Attacks.	30
4.1	Confusion Matrix of SNN model trained on DB2.	35
4.2	Confusion Matrix of SNN model trained on DB3.	36
4.3	Confusion Matrix of SNN model trained on DB4.	36
4.4	Confusion Matrix of SNN model trained on DB5.	37

List of Tables

2.1	Summary of IDS Methodologies, Contributions, and Dataset Usage	6
3.1	Attack category and subcategory-wise class distribution in Total DB1.	11
3.2	Feature Description of the Selected Features	13
3.3	Normal vs. Abnormal Ranges for Key Stateful Features and Their Attack Relevance	15
3.4	Reduced DB (DB2)	16
3.5	Class Distribution of DB3, DB4, and DB5	19
4.1	Evaluation Metrics of Models trained on DB2	33
4.2	Evaluation Metrics of Models trained on DB3	33
4.3	Evaluation Metrics of Models trained on DB4	34
4.4	Evaluation Metrics of Models trained on DB5	34

List of Abbreviations

CNN-GRU – Convolutional Neural Network – Gated Recurrent Unit	KNN – K-Nearest Neighbors
CSS – Cascading Style Sheet	LDA – Linear Discriminant Analysis
CSV – Comma Separated Values	LIF – Leaky Integrate-and-Fire
DB – Database	ML – Machine Learning
DDoS – Distributed Denial of Service	MitM – Man-in-the-Middle
DNS – Domain Name System	NIDS – Network Intrusion Detection System
DoS – Denial of Service	PKTS – Number of Packets
DT – Decision Tree	RA – Read Argus
DUR – Duration	RF – Random Forest
HTTP – Hypertext Transfer Protocol	RL – Reinforcement Learning
ICMP – Internet Control Message Protocol	SMOTE – Synthetic Minority Over-sampling Technique
IDS/IPS – Intrusion Detection System / Intrusion Prevention System	SNN – Spiking Neural Network
IoT – Internet of Things	SVM – Support Vector Machine
IP – Internet Protocol	TCP – Transmission Control Protocol
	UDP – User Datagram Protocol

Chapter 1

Introduction

1.1 Background and Motivation

The rapid growth of Internet of Things (IoT) across consumer, industrial, and enterprise environments has fundamentally reshaped the digital landscape. However, this vast interconnectedness creates an expansive and varied attack surface. Securing these resource-constrained devices is critical—a compromise can lead to serious data breaches, service disruptions, and even physical safety risks. Intrusion Detection Systems (IDS) serve as a vital defense layer, monitoring network traffic to identify and flag malicious activities in real-time. Effective IoT security frameworks must therefore be accurate, scalable, and efficient enough to operate within the constraints of modern edge computing environments.

1.2 Problem Statement

Despite the clear need for robust security, many IDS frameworks face significant practical challenges. A primary issue is the difficulty of achieving accurate, low-latency detection in a real-time environment. Furthermore, many benchmark security datasets, including the widely-used Bot-IoT dataset [1], are synthetically generated and suffer from a severe class imbalance. This imbalance leads to models that are heavily biased towards majority attack classes while performing poorly on minority classes. Critically, this includes the misclassification of scarce normal (benign) traffic as an attack (e.g., DDoS_TCP), a fatal flaw that renders them unreliable for real-world deployment.

1.3 Gaps in Existing Research

A review of existing studies reveals several open challenges. While many traditional machine learning classifiers report high offline accuracy on benchmark datasets, these results often do not translate to real-world, real-time deployment scenarios. These "base paper" approaches often fail to adequately address the class imbalance problem, leading to poor generalization and high false-positive rates for benign traffic. Moreover, there is a significant gap in the exploration of energy-efficient, event-driven neuromorphic models, such as Spiking Neural Networks (SNNs). These models are highly promising for the computational and power constraints of IoT edge devices but have not been sufficiently applied or evaluated in this domain.

1.4 Objectives of the Project

The primary objective of this project is to design, implement, and evaluate a robust, end-to-end intrusion detection system for IoT networks that addresses the gaps of class imbalance and real-time performance. The sub-objectives are as follows:

- To source and extensively preprocess the Bot-IoT dataset, creating multiple balanced and augmented versions (DB2-DB5) to systematically resolve class imbalance.
- To train and evaluate a suite of classical machine learning models (e.g., Decision Tree, Random Forest, SVM) and a Spiking Neural Network (SNN) across all dataset versions to measure the impact of data balancing.
- To build a full, real-time prediction pipeline integrating network packet capture (Argus) , concurrent data processing (Go) , and a message queuing system (Redis).
- To deploy the system in a containerized (Docker) environment that simulates edge devices and an external attack host to validate real-world performance.
- To implement a responsive visualization dashboard for the real-time monitoring of network alerts.

1.5 Proposed Methodology and Novelty

This project proposes a hybrid intrusion detection framework that leverages both classical ensemble classifiers and a Spiking Neural Network (SNN). The core novelty of this work is the

development of a complete, practical, end-to-end pipeline. In this system, live network traffic is captured by the **Argus** network audit tool and processed by its client (RA) to generate feature vectors. These vectors are fed into a high-concurrency aggregation service written in **Go** , which efficiently publishes them to a **Redis** pub-sub channel. A separate Python-based prediction service subscribes to these vectors, performs inference using the trained SNN model, and publishes the classification results. This entire architecture is deployed on **Docker** containers to simulate multiple edge devices, providing a realistic and scalable test environment.

1.6 Expected Results and Impact

The proposed framework demonstrates exceptional performance, particularly when using the SNN model trained on the custom-balanced **DB5 dataset**. This model achieves a robust and stable detection accuracy exceeding **96%**. This superior result is a direct consequence of our data augmentation strategy, which combined the **Synthetic Minority Over-sampling Technique (SMOTE)** for attack classes with 10,000 synthetically generated normal traffic vectors. This process successfully resolved the critical issue of normal traffic misclassification that plagued models trained on the original, imbalanced data. This work validates the framework as an adaptive, efficient, and scalable solution for modern IoT security, laying a strong foundation for deploying neuromorphic-assisted IDS in practical, real-world scenarios.

Chapter 2

Literature Survey

Recent years have witnessed rapid advancement in intrusion detection systems (IDS) tailored for Internet of Things (IoT) networks, prompted by heightened cybersecurity risks and the expanding IoT landscape. In particular, methodologies leveraging machine learning, ensemble strategies, and neuromorphic computing—including Spiking Neural Networks (SNN)—have become focal points for addressing key challenges such as data imbalance, energy efficiency, and real-time detection. This literature survey systematically reviews prominent research contributions, critical datasets like Bot-IoT, and evolving hybrid model architectures in IDS development.

The base paper by Alosaimi et al. (2023) lays a cornerstone in IDS research using the Bot-IoT dataset, proposing an ML-based multi-level intrusion detection system. Their comprehensive framework encompasses data preprocessing, balancing via SMOTE, and combining multiple ML classifiers including decision trees and ensemble bagging, yielding near-perfect accuracy exceeding 99.8% [2]. This work substantially contributes to practical IoT security by addressing the critical class imbalance in IoT traffic while showcasing the potential of ensemble classifiers in botnet detection. However, the study does not explore neuromorphic approaches and relies predominantly on synthetic lab data which may limit generalization.

Complementing this, Wang et al. (2024) investigate the efficacy of Spiking Neural Networks for intrusion detection, emphasizing their advantage in temporal data encoding and energy-efficient, biologically-inspired event-driven computation. Their model, benchmarked on datasets such as NSL-KDD and AWID, consistently outperforms classical ML and deep learning models in

accuracy and computational efficiency, highlighting the promise of SNN for resource-constrained IoT scenarios [3]. Nonetheless, limitations around real-time adaptive learning remain unaddressed, presenting opportunities for future extensions.

Nicho et al. (2023) focus on detecting IoT botnet attacks via classical ML algorithms applied to the Bot-IoT dataset, revealing significant challenges imposed by data imbalance, particularly in minority class detection. While majority classes yield high accuracy, the minority and normal classes suffer performance degradation, motivating the integrative balancing and hybrid modeling approaches pursued in this project [4].

Rahman et al. (2025) provide an encompassing survey of IDS approaches targeted at IoT networks, delineating taxonomy frameworks and resource-related constraints. While offering a broad perspective, their survey lacks coverage on neuromorphic paradigms such as SNN and does not delve into deployment complexities in edge environments [5]. Similarly, Sharma et al. (2025) review AI-driven IDS techniques emphasizing supervised, unsupervised, and reinforcement learning models, but do not address advanced neuromorphic computations [6].

Amine et al. (2025) introduce a hybrid deep learning model combining convolutional and gated recurrent units (CNN-GRU) optimized for IoT IDS. Their framework achieves superior detection accuracy on synthetic datasets by capturing both spatial and temporal attack characteristics. However, it is constrained by a lack of validation on real-world heterogeneous IoT networks [7]. This limitation resonates in Qaddos et al. (2024) who develop a similar hybrid CNN-GRU framework but on synthetic attack datasets, advocating the need for benchmarking on established public datasets such as Bot-IoT or NSL-KDD [8].

Manivannan et al. (2024) and Alghanam et al. (2023) reflect growing trends in ensemble learning and feature selection techniques for IoT IDS, achieving improvements in classification accuracy and minority class handling. Studies by Thakkar et al. (2023) and Douiba et al. (2023) extend this by integrating ensemble and boosting algorithms, further improving detection performance on imbalanced IoT traffic [9, 10, 11, 12].

Yarga and Wood (2024), along with PM Rahman et al. (2023), contribute foundational research on SNN methodologies. Yarga et al. demonstrate event-driven deep SNN architectures capable of dynamic anomaly detection, while PM Rahman et al. analyze theoretical model limitations and biological plausibility of SNNs, underscoring challenges including robustness in realistic

deployments [13, 14].

Lastly, Alotaibi and Mishra (2024) discuss bot data collection and cyber security IDS measures via deep learning in IoT. Their work adds complementary perspectives on data acquisition and hybrid model training that support evolving IDS frameworks [15].

2.0.1 Summary

The reviewed literature can be grouped into four methodological clusters for effective synthesis as shown in Table 2.1

Table 2.1: Summary of IDS Methodologies, Contributions, and Dataset Usage

Group	Papers Included	Contribution Approach	&	Datasets Used	Limitations/Gaps Identified
Bot-IoT Dataset- Based ML IDS	[2, 4, 10]	ML-based level IDS, techniques, methods	multi- balancing ensemble	BoT-IoT variants	Generalization limits, neuromorphic exclusion
Hybrid Deep Learning Techniques	[7, 8, 6]	CNN-GRU DL for spatial-temporal feature learning	and hybrid multi-IoT	Synthetic, multi-IoT datasets	Synthetic bias, lack public benchmarking
Neuromorphic SNN Approaches	[3, 13, 14]	SNN models for efficient anomaly detection		NSL-KDD, AWID, simulation datasets	Real-time adaptation gap, deployment complexity
Ensemble & Boosting Enhanced Models	[9, 11, 12]	Ensemble and boosted models, selection improvement	feature	BoT-IoT	Scalability and minority adaptation challenges

Despite advances, challenges persist including class imbalance, energy efficiency in edge deployments, real-time learning, and adaptation to unseen attack types. The present project addresses these

by integrating SNN models with ensemble classifiers, leveraging data balancing, and validating in containerized edge scenarios, thus providing a significant contribution towards holistic, scalable, and energy-efficient IoT intrusion detection.

2.0.2 Key Takeaways

The literature survey demonstrates strong advancements in intrusion detection for IoT, including the use of hybrid deep learning and neuromorphic models. Despite this, successful and practical deployment is still limited by issues such as class imbalance in datasets, poor detection rates for minority and normal traffic, and the challenges of scaling solutions to resource-constrained, real-world networks. Researchers highlight that combining classical and neuromorphic approaches improves both detection accuracy and efficiency, while advanced balancing and data augmentation techniques significantly enhance the handling of hard-to-detect cyber threats.

2.0.3 Open Challenges

Significant gaps persist in IoT security research. Most notably, dataset imbalance leads to unreliable detection of minority and normal classes, and the reliance on synthetic benchmarks limits a model's real-world generalizability. Energy and latency restrictions further complicate real-time operation at the edge, and practical neuromorphic or hybrid models are still under-explored for scalable, adaptive intrusion detection. This project addresses several of these barriers by integrating Spiking Neural Networks, advanced dataset balancing, and live edge deployment, but the field continues to face complex, evolving threats that will require ongoing innovation.

Chapter 3

System Design

This section presents the design and implementation of the intrusion detection system. It covers dataset sourcing and preparation, labeling and preprocessing, multi-model training with Spiking Neural Networks, a real-time prediction pipeline using containerized edge devices, and a responsive visualization dashboard. The accompanying block diagram summarizes the system architecture, showing data flow from network traffic generation through detection to alert visualization. Figure 3.1 illustrates the sequential workflow from dataset acquisition to real-time network monitoring and alert visualization as a block diagram.

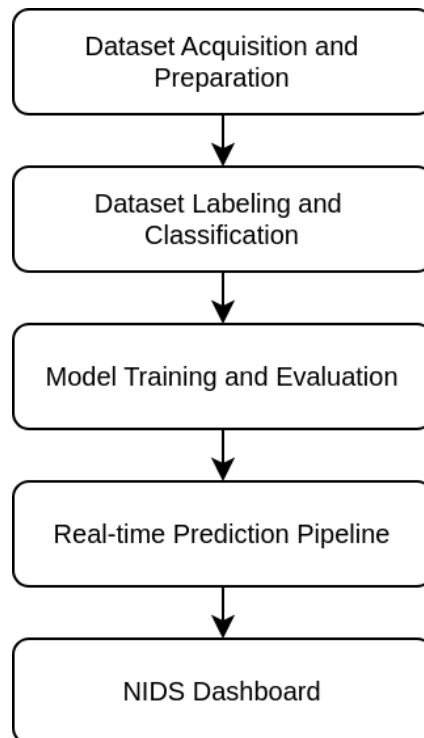


Figure 3.1: System workflow of the Intrusion Detection Framework.

Before delving into the specific components and implementation details of the proposed intrusion detection system, it is important to first clarify some foundational concepts underlying the overall design. A clear understanding of the core elements—such as edge devices, the broader IoT ecosystem, and the types of cyber threats addressed—sets the stage for exploring how each layer of the system contributes to robust real-time network security and efficient data handling at the edge.

3.1 Background

3.1.1 Edge Devices

Edge devices are hardware at the network boundary, acting as intermediaries between IoT devices and the cloud. They process data locally, reducing latency and bandwidth usage while enabling faster decisions. They handle data collection, filtering, routing, processing, storage, and security. Examples include routers, switches, firewalls, sensors, gateways, wearables, cameras, and hubs. They send traffic—sensor data, signals, alerts, and aggregated information—to central systems for analysis.

In IoT networks, edge devices connect and manage sensors and actuators across smart homes, industrial, and healthcare settings. Local processing reduces congestion, improves response times, and enhances scalability. Traffic includes telemetry, video streams, status updates, and commands—vital for operations and security. This distributed approach improves performance and resilience while supporting analytics and security in IoT architectures.

3.1.2 Attacks on Edge Devices

Edge devices in IoT networks are targeted by various cyberattacks exploiting their vulnerabilities. These attacks range from physical tampering to sophisticated software exploits that aim to disrupt services, steal data, or hijack device functions. Understanding these attack types is crucial for securing IoT infrastructures, especially at the edge where devices operate with limited computational resources and are often exposed to direct or network-based threats. Some type of attacks that are performed on edge devices are mentioned below:

- **Denial of Service (DoS) and Distributed Denial of Service (DDoS) Attacks:** These attacks overwhelm devices or networks with excessive traffic to interrupt or shut down

legitimate services.

- **Reconnaissance Attacks:** Include OS fingerprinting and port scanning, where attackers probe devices to gather information for future exploitation.
- **Data Exfiltration:** Unauthorized transfer of sensitive data from the device or network to malicious actors.
- **Keylogging:** Recording of keystrokes to capture sensitive user inputs like passwords or commands.
- **Physical Attacks:** Direct tampering or destruction of hardware components to compromise device function or extract data.
- **Firmware Attacks:** Exploiting vulnerabilities in the device firmware to install malicious code, gain persistent access, or disrupt operations.
- **Man-in-the-Middle (MitM) Attacks:** Intercepting and potentially altering communications between devices and servers.
- **Credential Attacks:** Brute-force and credential stuffing attacks exploiting weak or default passwords to gain unauthorized access.
- **Malware and Botnet Attacks:** Using malware to control devices for malicious purposes such as launching large-scale DDoS attacks.
- **Replay Attacks:** Resending previously captured legitimate data packets with malicious intent to deceive devices.

This project focuses on assisting the prediction of key cyber attacks in IoT and edge computing environments by employing advanced machine learning and neuromorphic models. The system is designed to identify and classify attacks such as Denial of Service (DoS), Distributed Denial of Service (DDoS), Reconnaissance, Data Exfiltration, and Keylogging in real-time.

3.2 Data Acquisition and Preperation

This section describes how the dataset was sourced, combined, and structured for the proposed network intrusion detection system. Starting with the pre-processed Bot-IoT dataset in CSV format, the process covers aggregating category-specific files into a single dataset, analyzing

class distributions, and selecting relevant features for modeling. Figure 3.2 illustrates the block diagram of data acquisition and preparation.

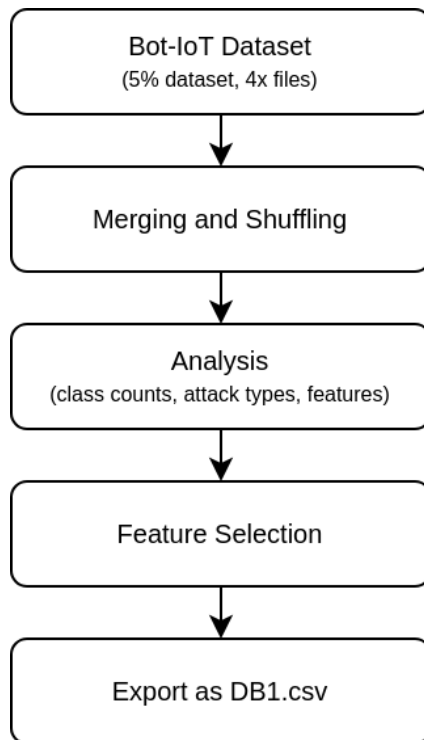


Figure 3.2: Block diagram of Data Acquisition and Preparation.

3.2.1 Bot-IoT Dataset Overview

Table 3.1: Attack category and subcategory-wise class distribution in Total DB1.

Attack	Category	Subcategory	Class Distribution	Total DB1(DB1)
Attack	DDoS	HTTP	989	1926624
		TCP	977380	
		UDP	948255	
	DoS	HTTP	1485	1650260
		TCP	615800	
		UDP	1032975	
	Reconnaissance	OS Fingerprint	17914	91082
		Service Scan	73168	
	Theft	Data Exfiltration	6	79
		Key Logging	73	
Total Attack Samples				3668045
Normal	Normal	Normal	477	477
Total				3668522

The Bot-IoT dataset was created at the Cyber Range Lab, UNSW Canberra, to simulate a realistic enterprise network with both legitimate and botnet traffic. The source data is available in multiple formats: pcap files (69.3 GB, 72 million records), Argus flow files, and CSV-formatted traffic summaries (16.7 GB). Attack types include DDoS (TCP/UDP/HTTP), DoS (TCP/UDP/HTTP), OS/Service Scanning, Keylogging, and Data Exfiltration—all organized by category and subcategory to simplify labeling.

For this implementation, the **pre-processed CSV subset provided by the authors (5% sample, 3 million records)** was used directly. The Table 3.1 shows the attack category and subcategory-wise class distribution in Total DB1.

3.2.2 Combined Dataset Creation

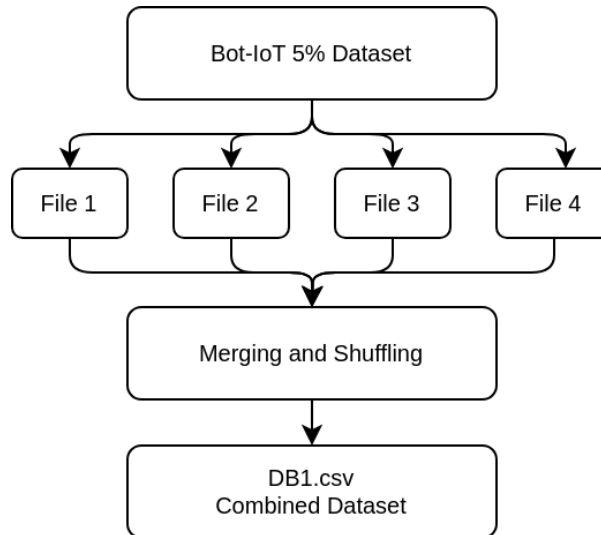


Figure 3.3: Block diagram of Combined Dataset Creation.

The 5% sample, originally split into four category-based CSV files (total 1.07 GB), was downloaded from the Bot-IoT repository. Python’s Pandas library was used to read and concatenate these files into a unified dataset, referenced as DB1.csv (Combiner Dataset). No database or MySQL was involved in this process—CSV file naming conventions were adopted for clarity during iterative dataset creation. This combined dataset forms the foundation for all downstream preprocessing and modeling steps. This process is illustrated as a block diagram in Figure 3.3.

This section covered the acquisition and preprocessing of the Bot-IoT dataset, feature selection, and construction of a unified target label for multi-class classification. The next section focuses

on dataset sampling, preprocessing routines—including scaling and encoding—and the iterative creation of multiple dataset versions to improve model accuracy and robustness.

3.2.3 Feature Selection

Selecting the right features is critical for optimizing real-time intrusion detection. Features were chosen based on their informational value, relevance to network flow analysis, and computational efficiency. This eliminates redundancy and derived columns that don't add predictive power. All categorical attributes (flags, protocol, state) were numerically encoded for streamlined processing. Derived statistics (mean, stddev, sum, min, max) were excluded, features like pkktSeqID, seq, saddr, daddr were dropped—they don't improve generalization and add unnecessary computational cost in real-time applications.

Beyond numerical features, the dataset uses a hierarchical target labeling scheme with three predictive levels:

- **Level 1:** Binary classification—normal (benign) versus attack traffic.
- **Level 2:** Broad attack categories, such as DoS, DDoS, Reconnaissance, and Theft.
- **Level 3:** Specific attack subcategories, including protocol-based attacks (TCP, UDP, HTTP), keylogging, and data exfiltration.

To simplify model training and deployment, category and subcategory labels are merged into a single target variable. This enables multi-class classification at both Level 2 and Level 3, reducing complexity while maintaining the granularity needed for effective intrusion detection.

The following Table 3.2 lists the retained features with descriptions.

Table 3.2: Feature Description of the Selected Features

Feature	Description
flgs_number	Numerical representation of flow state flags seen in transactions
proto_number	Numerical representation of transaction protocols present in network flow
Continued on next page	

Table 3.2 – continued from previous page

Feature	Description
Sport	Source port number
Dport	Destination port number
Pkts	Total count of packets in transaction
Bytes	Total number of bytes in transaction
state_number	Numerical representation of transaction state
Dur	Record total duration
Spkts	Source-to-destination packet count
Dpkts	Destination-to-source packet count
Sbytes	Source-to-destination byte count
Dbytes	Destination-to-source byte count
Rate	Total packets per second in transaction
Srate	Source-to-destination packets per second
Drate	Destination-to-source packets per second
TnBPSrcIP	Total number of bytes per source IP
TnBPDstIP	Total number of bytes per destination IP
TnP_PSrcIP	Total number of packets per source IP
TnP_PDstIP	Total number of packets per destination IP
TnP_PerProto	Total number of packets per protocol
TnP_Per_Dport	Total number of packets per destination port
AR_P_Proto_P_SrcIP	Average rate per protocol per source IP (calculated by pkts/dur)
AR_P_Proto_P_DstIP	Average rate per protocol per destination IP
N_IN_Conn_P_SrcIP	Number of inbound connections per source IP
N_IN_Conn_P_DstIP	Number of inbound connections per destination IP
AR_P_Proto_P_Sport	Average rate per protocol per source port
AR_P_Proto_P_Dport	Average rate per protocol per destination port
Pkts_P_State_P_Protocol_P_DestIP	Number of packets grouped by state of flows and protocols per destination IP

Continued on next page

Table 3.2 – continued from previous page

Feature	Description
Pkts_P_State_P_Protocol_P_SrcIP	Number of packets grouped by state of flows and protocols per source IP
Category_Subcategory	Traffic category and subcategory combined

Stateful features capture traffic patterns and aggregate behaviors that are sensitive indicators of network health. Table 3.3 summarizes typical threshold-based ranges for key features, differentiating normal IoT operation from suspected attacks. These boundaries guide both detection logic and downstream model training, ensuring that deviations rapidly trigger further investigation. By identifying and codifying abnormal behavior, feature selection directly improves detection accuracy and system robustness in dynamic IoT environments.

Table 3.3: Normal vs. Abnormal Ranges for Key Stateful Features and Their Attack Relevance

Feature	Normal Range	Abnormal Range
TnPPSrcIP	100 – 500 packets/min	$\geq 10,000$ packets/min
TnPPDstIP	100 – 500 packets/min	$\geq 10,000$ packets/min
TnBPSrcIP	1 MB – 10 MB per session	≥ 100 MB per session
TnBPDstIP	1 MB – 10 MB per session	≥ 100 MB per session
AR_P_Proto_P_SrcIP	Stable low average rate	Sudden spikes or bursts
AR_P_Proto_P_DstIP	Stable low average rate	Sudden spikes or bursts
N_IN_Conn_P_SrcIP	1-10 inbound connections/min	≥ 100 inbound connections/min
N_IN_Conn_P_DstIP	1-10 inbound connections/min	≥ 100 inbound connections/min

3.3 Dataset Labeling and Classification Setup

This section describes how classification-ready datasets were constructed by generating target labels, applying preprocessing steps, and creating multiple dataset versions for model training and evaluation.

3.3.1 Creation of DB2

DB2 was created by randomly sampling DB1, reducing the dataset from approximately 3.7 million to 63,030 records. Sampling was stratified—capping large classes at 10,000 samples while retaining all samples from smaller classes. Target labels were consolidated into a new **label** column by combining category and subcategory fields (e.g., **DoS_TCP**), and the original label columns were removed. The Table 3.4 shows the class distribution of each label in DB2. This controlled, smaller dataset enabled efficient model training with reduced computational overhead.

Table 3.4: Reduced DB (DB2)

Label	Class Distribution
DDoS_HTTP	989
DDoS_TCP	10000
DDoS_UDP	10000
DoS_HTTP	1485
DoS_TCP	10000
DoS_UDP	10000
Reconnaissance_OS Fingerprint	10000
Reconnaissance_Service Scan	10000
Theft_Data Exfiltration	6
Theft_Key Logging	73
Normal	477
Total	63030

3.3.2 Data Preprocessing

DB2 was prepared for model training through three steps, this is also illustrated in Figure 3.4:

- Missing values were filled with zero to ensure all feature columns were numerical.
- Label encoding was applied to the **label** column using a **LabelEncoder**, which was saved for reuse during inference.
- Min-Max scaling normalized all numerical features to a 0–1 range using a **Min-Max**

Scaler. The scaler object was saved for real-time scaling during prediction.

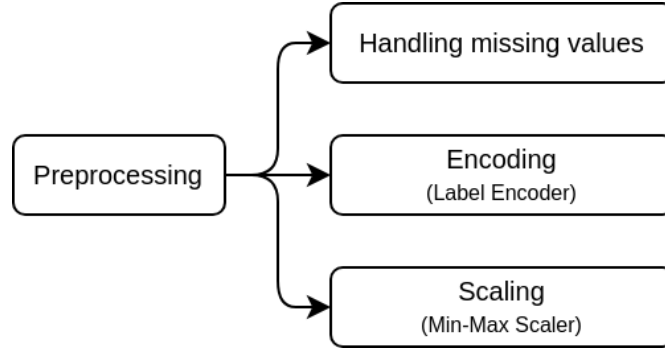


Figure 3.4: Data Preprocessing Steps.

Label Encoder

Label Encoding is a data transformation technique used to convert categorical text data into numeric form so that machine learning models can interpret and process it. Since most algorithms operate on numerical input, Label Encoding assigns each unique category an integer value. Equation 3.1 is used to convert categorical text columns—such as protocol type, service, or categorical labels.

Assume $C = \{c_1, c_2, \dots, c_n\}$ represents a categorical feature containing n unique categories. Label Encoding maps each category c_i to a unique integer y_i according to the mapping function:

$$f(c_i) = y_i, \quad \text{where } y_i \in \{0, 1, 2, \dots, n - 1\} \quad (3.1)$$

For example, if $C = \{\text{HTTP}, \text{TCP}, \text{UDP}\}$, then the encoder may assign $\text{HTTP} \rightarrow 0$, $\text{TCP} \rightarrow 1$, $\text{UDP} \rightarrow 2$.

Min-Max Scaler

Min-Max Scaling (also called normalization) rescales numeric features into a specific range, typically. This technique ensures uniformity across features by preventing attributes with larger numeric ranges from dominating those with smaller ranges. Normalization is essential for models sensitive to feature magnitude, such as neural networks, SVMs, and logistic regression. Equation 3.2 is applied to all continuous features so that their values are normalized before being passed to the learning algorithms.

For each feature x with minimum value $x_{\min}$ and maximum value $x_{\max}$, the Min-Max Scaler

transforms the data as:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (3.2)$$

where x' is the scaled value ranging between 0 and 1.

3.3.3 Subsequent Database Versions (DB3-DB5)

Additional dataset versions were created to address model limitations observed during training and prediction:

- **DB3:** Derived from DB2 using the Synthetic Minority Over-sampling Technique (SMOTE) to balance classes. Major classes were set to 10,000 samples, while minor classes received proportionally smaller targets. This improved balance reduced bias during model fitting. 3.3 is applied to generate new feature vectors for the minority class until the desired oversampling ratio is reached..
- **DB4:** Built by augmenting the scarce normal class with 10,000 synthetically generated (using Feature Vector Generation, refer Subsection 3.5.1) normal traffic vectors, compensating for the initially low count (477 rows). Other classes retained their original imbalance.
- **DB5:** Combined DB4's augmented normal class data with SMOTE balancing applied to attack classes—merging realistic normal traffic samples with balanced attack representation.

SMOTE

SMOTE is an advanced oversampling approach used to address class imbalance in datasets, especially where the minority class is underrepresented and thus poses challenges for most machine learning models. Unlike simple duplication of minority class samples, SMOTE generates new, synthetic data instances by interpolating between existing minority samples in the feature space.

For each minority sample x , SMOTE:

- Finds its k -nearest neighbors among other minority instances,
- Randomly picks one neighbor x_k ,

- Generates a new synthetic sample x' as:

$$x' = x + \delta \cdot (x_k - x) \quad (3.3)$$

where δ is a random number in the continuous interval $[0, 1]$.

- If $\delta = 0$, x' coincides with the original sample x .
- If $\delta = 1$, x' matches the neighbor x_k .
- Typically, δ is drawn anew for every synthesized point, spreading samples evenly along the line connecting x and x_k .

Table 3.5: Class Distribution of DB3, DB4, and DB5

Label	Class Distribution DB3	Class Distribution DB4	Class Distribution DB5
DDoS_HTTP	10000	989	10000
DDoS_TCP	10000	10000	10000
DDoS_UDP	10000	10000	10000
DoS_HTTP	10000	1484	10000
DoS_TCP	10000	10000	10000
DoS_UDP	10000	10000	10000
Reconnaissance_OS Fingerprint	10000	10000	10000
Reconnaissance_Service Scan	10000	10000	10000
Theft_Data Exfiltration	100	6	100
Theft_Key Logging	500	73	500
Normal	10000	10000	10000
Total	90600	72553	90600

The Table 3.5 shows the class distribution of each label in DB3, DB4, and DB5. Each dataset version underwent identical preprocessing: missing value handling, label encoding, and scaling. This maintained compatibility across experiments and improved classification robustness.

This section outlined the methods for labeling, preprocessing, and balancing the dataset, resulting in robust training sets. The next section presents the model training process, evaluation strategy, and iterative improvements based on observed performance.

3.4 Model Training and Evaluation

This section describes the training process for various classification models—including traditional machine learning classifiers and a Spiking Neural Network (SNN)—their evaluation using accuracy and classification reports, and the iterative refinements made to improve detection accuracy based on observed limitations.

3.4.1 Training and Evaluation Setup

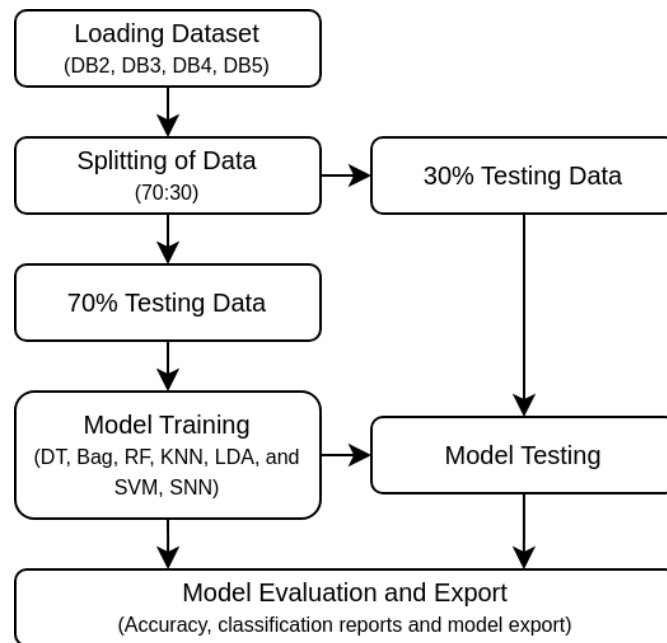


Figure 3.5: Flowchart of Model Training and Evaluation.

Several classical machine learning models were trained and evaluated on the processed datasets: Decision Tree (DT), Bagging Classifier (Bag), Random Forest (RF), K-Nearest Neighbors (KNN), Linear Discriminant Analysis (LDA), and Support Vector Machine (SVM). The dataset was split randomly into 70% training and 30% testing subsets while maintaining class stratification.

The SNN model was then implemented using the PyTorch framework and the snnTorch library to leverage neuromorphic computing principles. The SNN architecture used fully connected layers with Leaky Integrate-and-Fire (LIF) neuron models. Equation 3.4 governs how the

membrane potential evolves over time, integrating synaptic inputs and natural decay ("leak") between spikes. It was trained for 20 epochs with a batch size of 512. Standard scaling was applied to feature inputs, and label encoding was applied consistently across all models.

SNN

Spiking Neural Networks (SNNs) are a class of artificial neural networks that more closely mimic the information processing of the biological brain by communicating through discrete, time-dependent events known as spikes. SNNs introduce a temporal aspect to information processing, allowing the network to dynamically react to sequences and patterns over time, rather than relying only on static snapshot data.

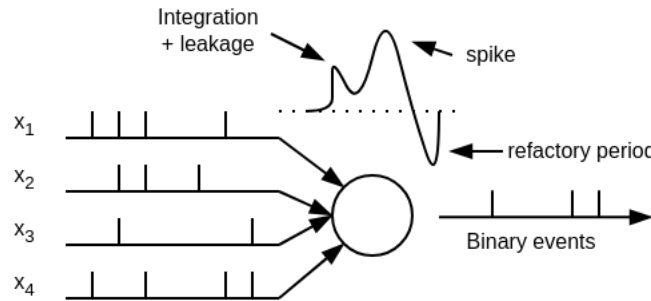


Figure 3.6: The Leaky-Integrate and Fire (LIF) model.

A commonly used mathematical formalism for SNNs is the Leaky Integrate-and-Fire (LIF) model, Figure 3.6 shows the illustration of the LIF model.

The Leaky Integrate-and-Fire (LIF) neuron model underpins the SNN architecture implemented in this work. Each input feature corresponds to a separate input channel, so with 26 selected features, the SNN receives 26 input streams. Over time, the LIF neuron integrates incoming spikes, with each spike incrementally increasing the neuron's membrane potential. At the same time, a leak mechanism ensures that this potential gradually decays if not continuously stimulated. Whenever the membrane potential exceeds a set threshold, the neuron emits a spike—interpreted as a binary firing event or decision. Immediately after spiking, the neuron resets and enters a refractory period during which further firing is temporarily inhibited. This operational dynamic is visually summarized in Figure 3.6, illustrating how multiple inputs (x_1 - x_4) are integrated, how leakage occurs, how spikes are generated, and the manifestation of refractory intervals in the output.

The state of each neuron, usually represented as its membrane potential $V(t)$, evolves according to the following differential equation:

$$\tau_m \frac{dV(t)}{dt} = -V(t) + RI(t) \quad (3.4)$$

where:

- τ_m is the membrane time constant,
- $V(t)$ is the membrane potential at time t ,
- R is the membrane resistance,
- $I(t)$ is the synaptic input current at time t .

A spike is emitted when $V(t)$ exceeds a threshold V_{th} :

$$\text{If } V(t) \geq V_{th} : \quad \text{neuron fires, and } V(t) \rightarrow V_{reset}$$

where V_{reset} is the reset voltage after the spike.

SNNs are typically structured in network layers similar to regular neural networks, but each neuron's output is a binary event (spike) which is passed to subsequent layers only when it occurs.

Models were evaluated using accuracy metrics and detailed classification reports, including class-wise precision, recall, and F1-score. The flowchart of classifier model training and evaluation is illustrated in Figure 3.5. The SNN model's confusion matrix was also visualized to analyze per-class performance patterns, this is discussed in-detail in Section 4.0.1.

3.4.2 Limitations and Iterations

Initial training on DB2 and DB3 revealed a significant limitation: normal traffic was frequently misclassified as DDoS_TCP in real-time predictions, despite high overall accuracy (99% for classical models, 92% for SNN) during testing. This issue likely stemmed from the limited number of normal samples (477 rows) in the Bot-IoT dataset, which were generated in a lab environment.

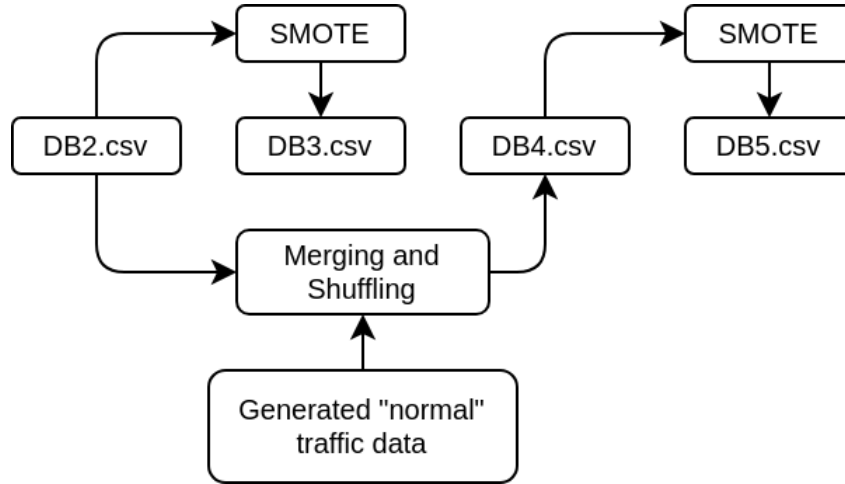


Figure 3.7: Flowchart of Creation of DB3, DB4 and DB5.

To address this, DB4 was created by adding 10,000 synthetically generated normal traffic vectors to better represent real-world conditions. DB5 then combined DB3’s class balancing with DB4’s enhanced normal samples. These changes substantially improved the model’s ability to correctly classify normal traffic in real-time scenarios, resolving the misclassification issue. The Figure 3.7 visualizes the process of creation of DB3, DB4 and DB5.

This section covered the training methodologies, models, and classification metrics. The normal traffic misclassification problem was identified and resolved through dataset augmentation and iterative refinement. The next section describes the real-time prediction pipeline that integrates these trained models for live intrusion detection and alerting.

3.5 Real-Time Prediction Pipeline

This section details the architecture of the real-time prediction pipeline, including edge device deployment using Docker for normal traffic generation, external host-based attack simulation, feature vector extraction with Argus and RA, concurrent data aggregation via a Go service, and live inference using the trained Spiking Neural Network model. The system architecture ensures scalable, asynchronous, and timely intrusion detection. Figure 3.8 illustrates and Real-Time Prediction Pipeline.

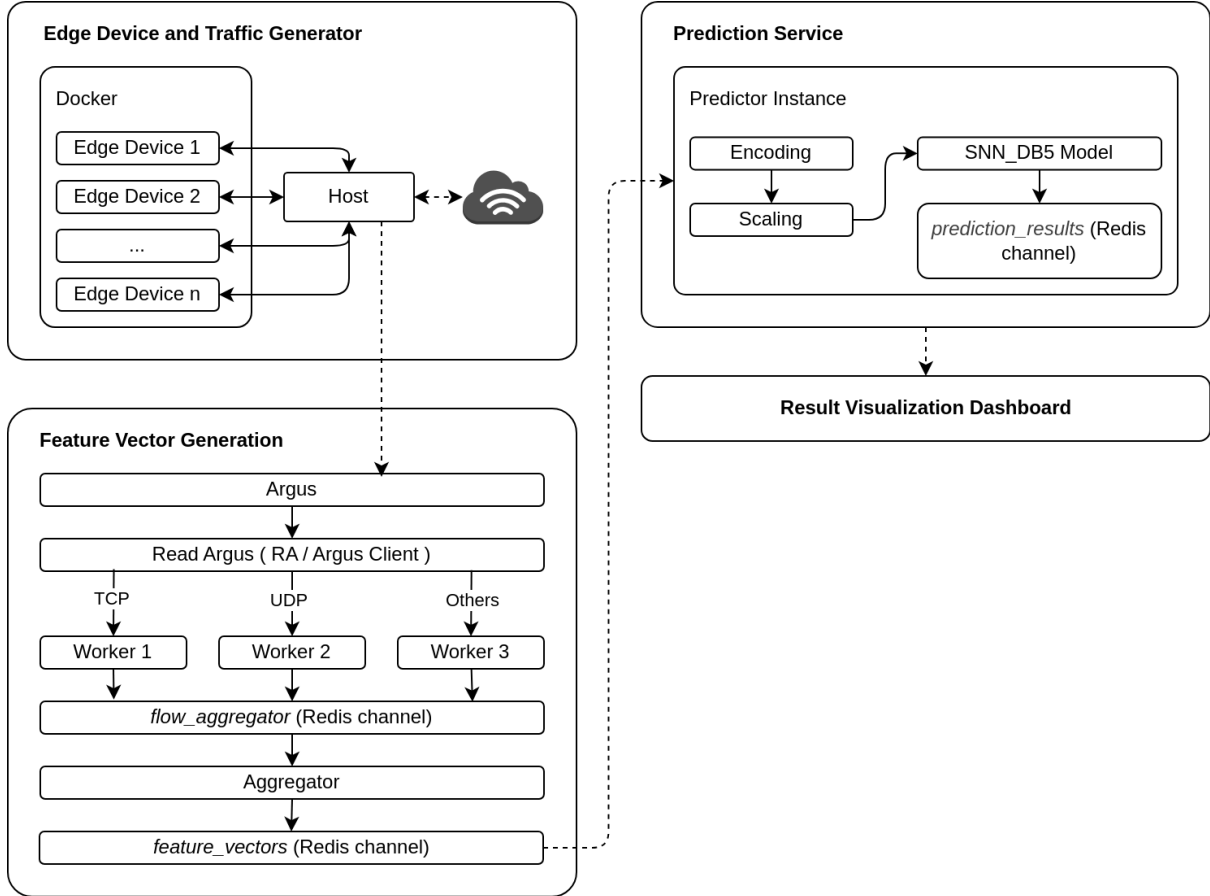


Figure 3.8: Block diagram of the Real-Time Prediction Pipeline.

3.5.1 Feature Vector Generation

The pipeline begins with **Argus**, a widely-used network audit tool. Developed in the early 1980s and deployed globally in cybersecurity applications, Argus processes live network traffic and generates flow records that capture essential communication metadata—inspired by telephone call record systems for comprehensive audit functionality aligned with NIST security controls. Each flow is uniquely identified using a five-element tuple, consisting of Source IP address, Destination IP address, Source port, Destination port, and Protocol (e.g., TCP, UDP, etc.). Whenever a device sends packets to another device, Argus groups those packets into a single flow if all 5-tuple values match.

For example, suppose Device A (IP: 192.168.1.10, Port: 54321) sends data to Device B (IP: 192.168.1.20, Port: 80) over TCP. Argus recognizes this as a single flow with the 5-tuple: (Source IP: 192.168.1.10, Destination IP: 192.168.1.20, Source Port: 54321, Destination Port: 80, Protocol: TCP).

Argus aggregates and summarizes these flows, providing records of activity between hosts. This process enables robust, scalable network monitoring and is a foundational step for accurate intrusion detection and audit throughout the system.

Argus feeds raw flow data to **Read Argus (RA / Argus Client)**, a command-line client for the Argus network flow monitoring system. It reads Argus data from standard input (e.g., piped directly from Argus), stored flow files, or remote Argus servers. RA filters, formats, and outputs network flow records based on user-defined criteria. It supports flexible selection of flow fields such as source/destination IP, ports, protocol, and packet counts. RA converts raw Argus audit flow records into usable data for analysis and feature extraction in network security workflows.

RA extracts key network features including source/destination IPs and ports (**sport**, **daddr**, **dport**), protocol type (**proto**), connection state (**state**), flags (**flgs**), timestamps (**stime**), duration (**dur**), and packet and byte counts from source and destination (**spkts**, **dpkts**, **sbytes**, **dbytes**)—all critical for intrusion assessment.

A custom **Argus preprocessor** written in **Go** handles this flow data with high concurrency. **Go** (also known as **Golang**) is a statically typed, compiled language designed by **Google** that emphasizes simplicity, efficiency, and robust concurrency handling. These characteristics make Go ideal for high-performance, parallel processing tasks like real-time network traffic analysis. The preprocessor operates in two modes:

Worker: Three workers monitor specific traffic classes:

- Worker 1 tracks TCP traffic, primarily DoS and DDoS attacks.
- Worker 2 focuses on UDP traffic, often containing DNS amplification and UDP flooding attempts.
- Worker 3 monitors ICMP and other protocols, such as ping floods and reconnaissance scans.

Each worker receives filtered Argus flow records, extracts relevant features, and publishes JSON messages to a Redis channel named **flow_aggregator**, Redis is an in-memory data structure store used as a message broker for real-time publish-subscribe data flow between detection modules and the frontend.

Aggregator: A separate process consumes messages from **flow_aggregator**, combines them, and processes them into comprehensive **feature vectors** for machine learning models. These vectors match the feature set described earlier and are published to the **feature_vectors** Redis channel.

Messages on the **flow_aggregator** channel contain individual flow data with fields including **saddr**, **sport**, **daddr**, **dport**, **proto**, **state**, **flgs**, **stime**, **dur**, **spkts**, **dpkts**, **sbytes**, and **dbytes**, as shown in Figure 3.9. The aggregator reads these records, applies transformations and aggregation, and emits feature vectors ready for inference. A feature vector message pushed into the **feature_vector** channel is displayed in Figure 3.10.

```

{
  "worker_id": 1,
  "flow": {
    "Saddr": "172.21.0.2",
    "Sport": 60314,
    "Daddr": "172.21.0.1",
    "Dport": 80,
    "Proto": "tcp",
    "State": "RST",
    "Flgs": " e          ",
    "Stime": " e          ",
    "Dur": 0,
    "Spkts": 0,
    "Dpkts": 1,
    "Sbytes": 1,
    "Dbytes": 74
  },
  "timestamp": 1761761298733784512
}

```

Figure 3.9: Message pushed into the flow_aggregator Redis channel.

```

{
  "AR_P_Proto_P_Dport": 0, "AR_P_Proto_P_DstIP": 0,
  "AR_P_Proto_P_Sport": 0, "AR_P_Proto_P_SrcIP": 0,
  "Daddr": "172.21.255.255", "N_IN_Conn_P_DstIP": 2,
  "Pkts_P_State_P_Protocol_P_DstIP": 2,
  "Pkts_P_State_P_Protocol_P_SrcIP": 2,
  "Saddr": "172.21.0.1", "Stime": " e          ",
  "TnBPdIP": 126, "TnBPSrcIP": 0,
  "TnP_PdIP": 2, "TnP_PSrcIP": 0,
  "TnP_PerProto": 2, "TnP_PerDport": 1,
  "bytes": 63, "dbytes": 63, "dpkts": 1,
  "dport": 32414, "drate": 0, "dur": 0,
  "flgs_number": 0, "pkts": 1, "proto_number": 17,
  "rate": 0, "sbytes": 0, "spkts": 0, "sport": 49954,
  "srate": 0, "state_number": 4
}

```

Figure 3.10: Message pushed into the feature_vectors Redis channel.

The pipeline uses Redis pub-sub for efficient queuing and asynchronous communication, enabling near real-time analysis with scalable concurrency.

3.5.2 Prediction Service

The prediction service subscribes to the **feature_vectors** Redis channel and continuously listens for new messages. Upon receiving a vector, the service preprocesses it using the training pipeline’s scaler and label encoder, then performs inference using the trained Spiking Neural Network model (trained on DB5).

After prediction, classification results—including the predicted label and timestamp are published to the **prediction_results** Redis channel, as shown in Figure 3.11. This separation of input and output streams enables decoupled, scalable pipeline components.



```
{
  "flow_info": { ...
  },
  "prediction": "Normal",
  "timestamp": 1761761333.4122465
}
```

Figure 3.11: Message pushed into the prediction_results Redis channel.

The prediction service’s responsive design enables immediate network intrusion detection alerts and integrates smoothly with downstream systems such as dashboards or alert managers.

This section established the architecture of the real-time prediction pipeline, highlighting Argus and RA for feature extraction, a Go-based aggregator for efficient flow data consolidation, and an SNN-powered prediction service for live network intrusion classification. The next section covers the design and implementation of the visualization dashboard and alert system.

3.5.3 Edge Device Deployment and Traffic Generation

The intrusion detection system generates real-time network traffic using multiple edge devices deployed as Docker containers. **Docker** is a leading containerization platform that creates, deploys, and runs applications in lightweight, portable containers. Docker containers encapsulate all dependencies, ensuring consistency across edge devices and simplifying deployment.

Approximately four edge devices run on the network, each with a unique IP address to simulate realistic scenarios. This containerized approach enables scalable and reproducible traffic generation while isolating the simulation environment and simplifying management.

These containers run a dedicated Python-based traffic generator script that produces continuous

streams of **normal network traffic**. This simulation encompasses a variety of legitimate traffic patterns to mimic real-world device behavior.

In contrast, **attack traffic is generated externally from the host Ubuntu machine** where the Docker containers are deployed. This host-based attack simulation includes various cyber-attack scenarios such as Distributed Denial of Service (DDoS) floods, protocol-specific exploits, reconnaissance scans, keylogging, and data exfiltration. Generating attacks on the host allows precise control over attack parameters and timing, simulating a realistic threat environment against the Dockerized edge devices.

Docker's lightweight virtualization ensures consistency across edge devices while maintaining resource efficiency. This enables rapid testing and validation of the intrusion detection system. The isolated containers serve as independent sources of network flows, allowing robustness testing under varied simulated conditions.

This section covered traffic generation, feature extraction, and neural inference for delivering live network attack predictions. The next section presents the visualization dashboard's design and functionality, which receives these predictions and provides analysts with interactive real-time monitoring, alerting, and investigative capabilities.

3.6 Result Visualization Dashboard

This section presents the architecture and user interface design of the network intrusion detection dashboard. The dashboard visualizes real-time alerts and network security metrics by subscribing to live Redis channels, providing security analysts with comprehensive situational awareness through interactive components. Figure 3.12 illustrates and Result Visualization Dashboard.

3.6.1 Dashboard Architecture

The dashboard is built using **Next.js**, a React framework emphasizing server-side rendering and seamless API integration, used here for dynamic dashboards with reactive state hooks with the modern App Router approach, **Tailwind CSS**, a utility-first CSS framework enabling rapid and customizable styling, and **TypeScript**, a superset of JavaScript providing static type checking to reduce runtime errors and improve maintainability. This stack delivers enhanced developer productivity, strong typing guarantees, and fast, responsive UI rendering.

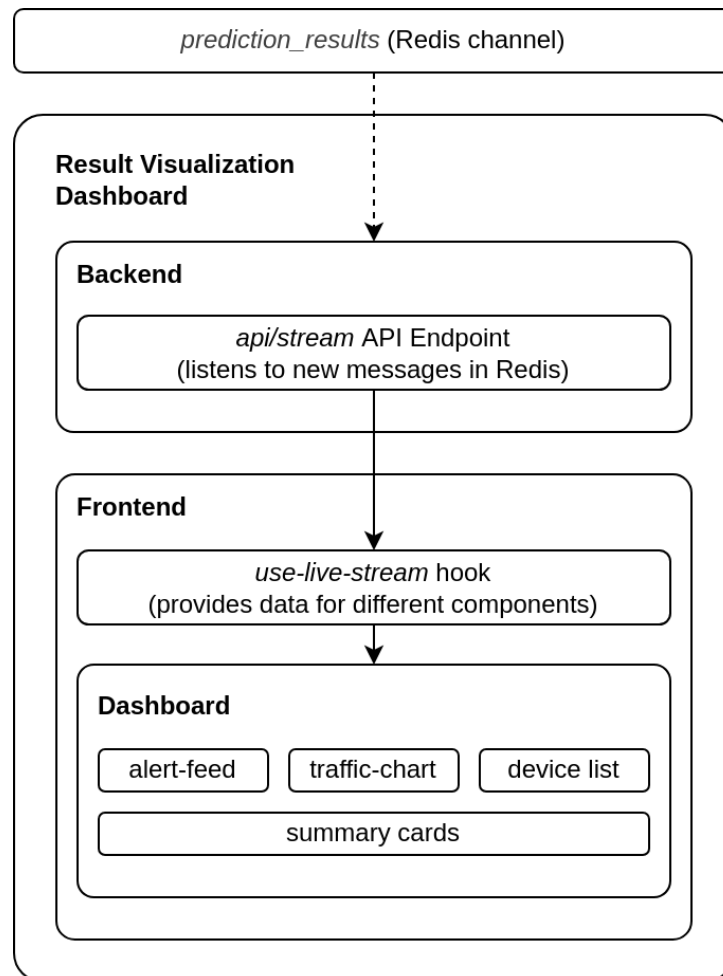


Figure 3.12: Architecture of the Result Visualization Dashboard.

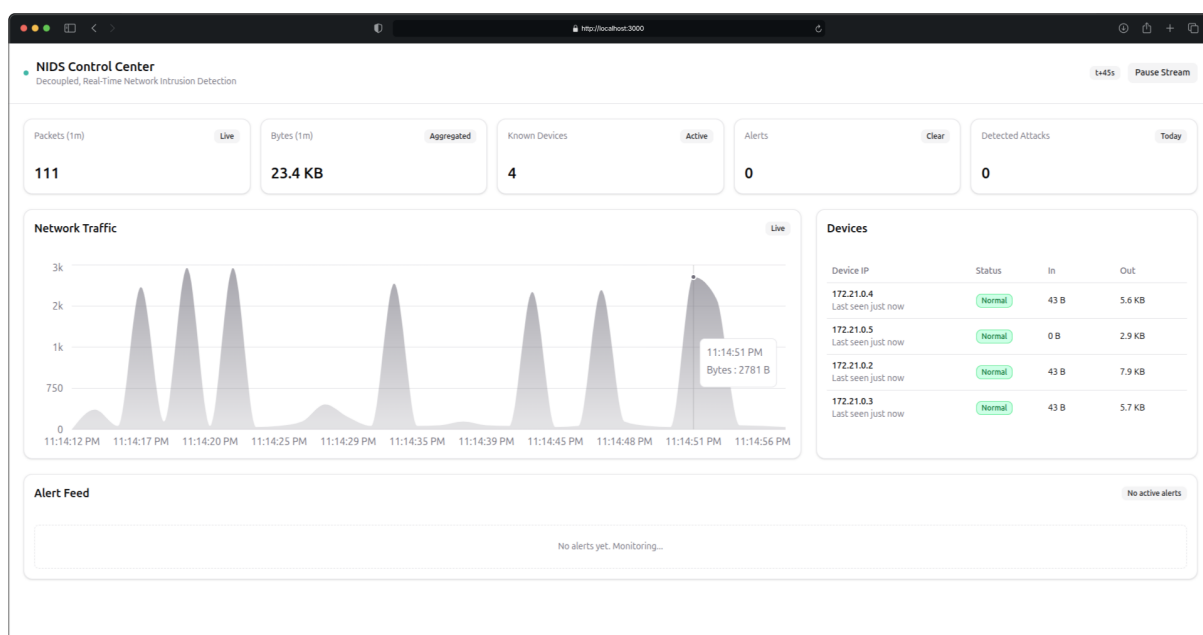


Figure 3.13: Mockup of the Dashboard with no Attacks.

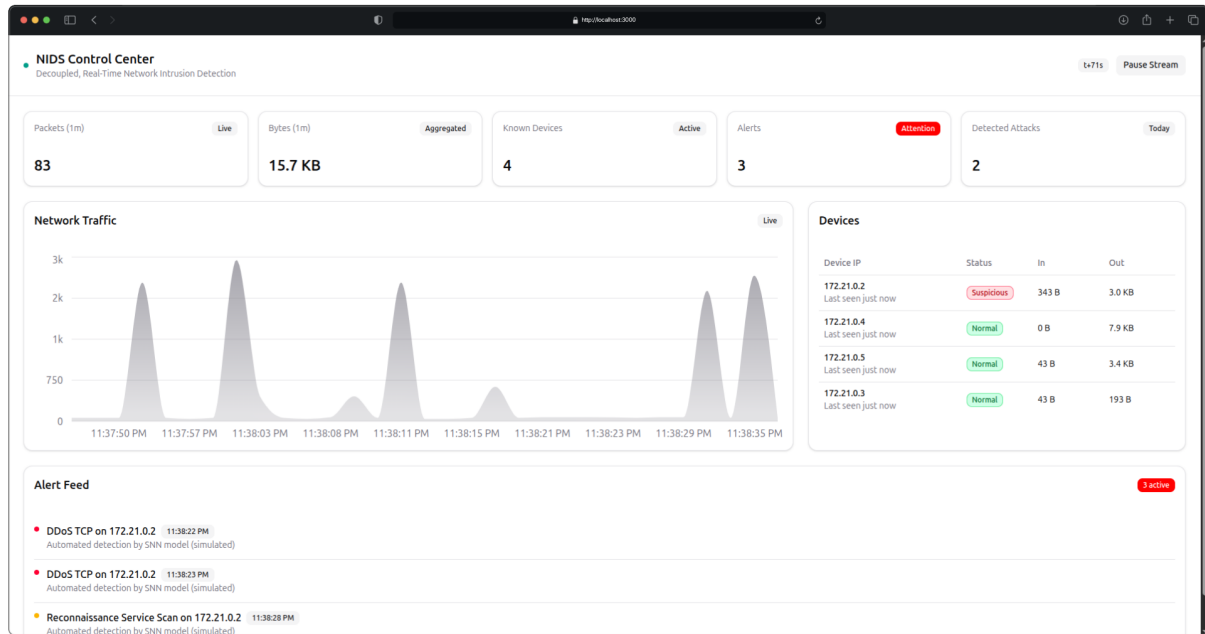


Figure 3.14: Mockup of the Dashboard with Attacks.

For real-time data integration, the dashboard connects to the backend through a custom API route (`api/stream`) that subscribes to the Redis `prediction_results` channel. New prediction messages arrive as streaming data and are processed via React hooks that manage live updates and state synchronization.

3.6.2 User Interface Components

The dashboard includes the following key interactive components for network security monitoring, Figure 3.13 and Figure 3.14 shows the dashboard containing the mentioned components with and without attacks:

- **Live Alert Feed:** Displays real-time intrusion alerts with timestamps, attack categories, source/destination IPs, protocols, and severity levels—color-coded for quick threat identification.
- **Traffic Charts:** Interactive time-series graphs showing traffic trends and detected attacks over selectable intervals, implemented via React components that encapsulate Chart.js, a JavaScript charting library used for rendering responsive, interactive time-series line and bar charts for visual analytics in security monitoring..
- **Device List Panel:** Lists connected network devices and their associated security events, with filtering and search capabilities for focused investigation.

- **Summary Cards:** Provide aggregate metrics such as total attacks detected, active alerts, and attack type distributions at a glance.
- **Real-Time Data Streaming Hook:** A custom React hook manages WebSocket (or Server-Sent Events) connections to maintain a live stream of updates from the backend.

The dashboard's modular component architecture enables scalability and extensibility, allowing future additions like custom filters, export functions, and enhanced alert workflows.

This section established the foundational components and workflows of the proposed intrusion detection system, including end-to-end data handling, model training, real-time prediction, and result visualization. The next section will analyze experimental results, provide performance metrics, and discuss model efficacy and limitations in practical scenarios.

Chapter 4

Results and Discussion

This section evaluates the developed models and demonstrates their effectiveness in detecting IoT network attacks. We discuss performance metrics, confusion matrices, real-time prediction analysis, and comparative results against existing studies to provide insight into each model’s capabilities and limitations.

4.0.1 Model Performance Evaluation

This section presents the performance of all trained models across database versions DB2 to DB5. The models include Decision Tree (DT), Bagging, Random Forest (RF), K-Nearest Neighbors (KNN), Linear Discriminant Analysis (LDA), Support Vector Machine (SVM), and Spiking Neural Network (SNN). Performance metrics cover accuracy, precision, recall, and F1-score weighted averages, along with class-wise breakdowns. Key observations from the confusion matrices are also highlighted.

Evaluation of models trained on DB2

On DB2, tree-based models (DT, Bagging, RF) achieve perfect 1.00 precision, recall, and F1-scores with fast training times. KNN also performs well at 95% across metrics. However, LDA and SVM underperform (F1: 0.59 and 0.70) due to class imbalance and non-linear patterns. SNN achieves 93% accuracy and F1-score but shows lower recall for normal traffic, often misclassifying normal samples as DDoS_TCP or DDoS_HTTP. This highlights challenges from synthetic imbalance and insufficient realistic normal samples—addressed in later database versions. The Table 4.1 shows the evaluation metrics of models trained on DB2.

Table 4.1: Evaluation Metrics of Models trained on DB2

Model	Accuracy	Precision	Recall	F1-Score	Time (s)
Decision Tree Classifier	0.998	1	1	1	0.3897
Ensemble Bagging Classifier	0.9983	1	1	1	3.3444
Random Forest Classifier	0.999	1	1	1	1.6932
K-Nearest Neighbors	0.9455	0.95	0.95	0.95	1.1118
Linear Discriminant Analysis	0.6136	0.67	0.61	0.59	0.1376
Support Vector Machine	0.7158	0.75	0.72	0.7	364.2769
Spiking Neural Network	0.9327	0.94	0.93	0.93	45.5894

Evaluation of models trained on DB3

DB3 used SMOTE for balancing. Tree-based models (DT, Bagging, RF) achieved perfect 1.00 scores. KNN improved significantly to 96%, mitigating bias. While LDA and SVM underperformed (F1: 0.60, 0.74), the SNN model benefited greatly, reaching 94% accuracy and F1-score. The remaining errors highlight that SMOTE improves balance but not the inherent diversity of rare attacks. The Table 4.2 shows the evaluation metrics of models trained on DB3.

Table 4.2: Evaluation Metrics of Models trained on DB3

Model	Accuracy	Precision	Recall	F1-Score	Time (s)
Decision Tree Classifier	0.9982	1	1	1	1.0804
Ensemble Bagging Classifier	0.9987	1	1	1	7.2497
Random Forest Classifier	0.999	1	1	1	3.9382
K-Nearest Neighbors	0.9611	0.96	0.96	0.96	2.3949
Linear Discriminant Analysis	0.6062	0.66	0.61	0.6	0.223
Support Vector Machine	0.7547	0.8	0.75	0.74	693.8833
Spiking Neural Network	0.937	0.94	0.94	0.94	63.9671

Evaluation of models trained on DB4

DB4 augmentation of the normal class significantly improves all models' handling of benign traffic. SNN and KNN show the most notable gains, with F1-scores rising to 0.96 and 0.94.

Tree-based models achieve perfect 1.00 scores. This marks a key turning point: misclassification of normal flows as attacks drops drastically. LDA performance jumps to 0.77 F1, and SVM’s F1 climbs to 0.83. Overall, DB4 demonstrates that realistic, sufficiently sampled benign data is essential for robust intrusion detection and real-world generalization. The Table 4.3 shows the evaluation metrics of models trained on DB4.

Table 4.3: Evaluation Metrics of Models trained on DB4

Model	Accuracy	Precision	Recall	F1-Score	Time (s)
Decision Tree Classifier	0.998	1	1	1	0.4593
Ensemble Bagging Classifier	0.9984	1	1	1	3.4666
Random Forest Classifier	0.999	1	1	1	1.8228
K-Nearest Neighbors	0.9422	0.94	0.94	0.94	0.9974
Linear Discriminant Analysis	0.7734	0.79	0.77	0.77	0.1729
Support Vector Machine	0.8303	0.86	0.83	0.83	232.1613
Spiking Neural Network	0.9578	0.96	0.96	0.96	50.592

Evaluation of models trained on DB5

Table 4.4: Evaluation Metrics of Models trained on DB5

Model	Accuracy	Precision	Recall	F1-Score	Time (s)
Decision Tree Classifier	0.9983	1	1	1	0.5989
Ensemble Bagging Classifier	0.9989	1	1	1	4.2821
Random Forest Classifier	0.9991	1	1	1	2.4409
K-Nearest Neighbors	0.9538	0.95	0.95	0.95	1.3682
Linear Discriminant Analysis	0.76	0.78	0.76	0.75	0.1963
Support Vector Machine	0.8501	0.87	0.85	0.85	318.0939
Spiking Neural Network	0.9617	0.96	0.96	0.96	64.3562

DB5 combines SMOTE-balanced attack classes with augmented normal samples, achieving the most balanced performance. Ensemble models reach near-perfect scores. KNN stabilizes at 95%, LDA at F1 0.75, and SVM at 0.85. SNN achieves its best results—96% accuracy and F1-score—handling major attacks and normal flows with minimal misclassifications. Weaknesses

persist only in the rarest classes (keylogging and data exfiltration). Minority and majority classes are now well supported, evidenced by confusion matrix diagonals and class-wise reports. This version offers the best foundation for live deployment, with optimized predictive performance and class-wise reliability for real-world network security. The Table 4.4 shows the evaluation metrics of models trained on DB5.

Classification Matrix of SNN model trained on various DBs

The SNN confusion matrices across DB2 to DB5 show the model's improved ability to distinguish between attacks and normal traffic as balancing strategies evolved. Figures 4.1, 4.2, 4.3 and 4.4 show the confusion matrices of the SNN model trained on various datasets (DB2, DB3, DB4 and DB5).



Figure 4.1: Confusion Matrix of SNN model trained on DB2.

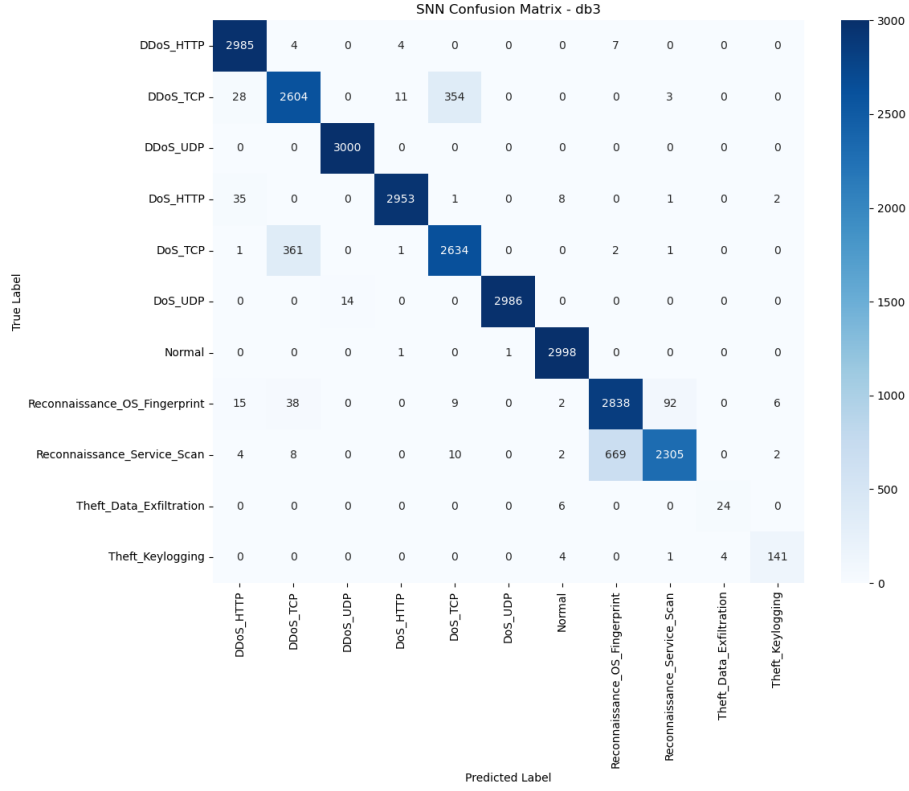


Figure 4.2: Confusion Matrix of SNN model trained on DB3.

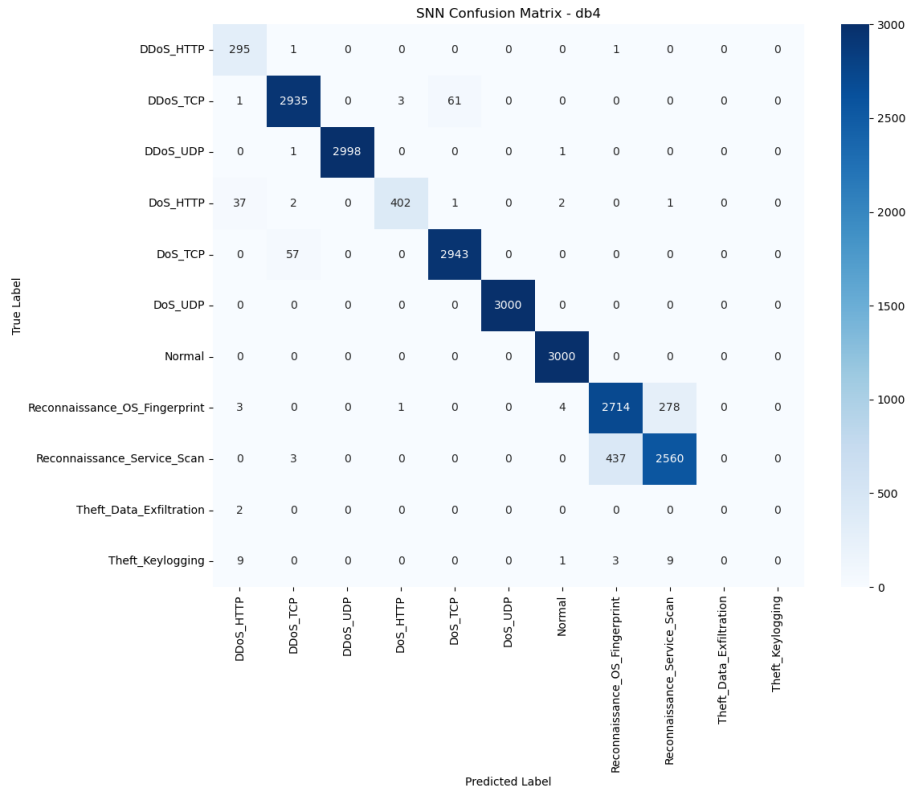


Figure 4.3: Confusion Matrix of SNN model trained on DB4.

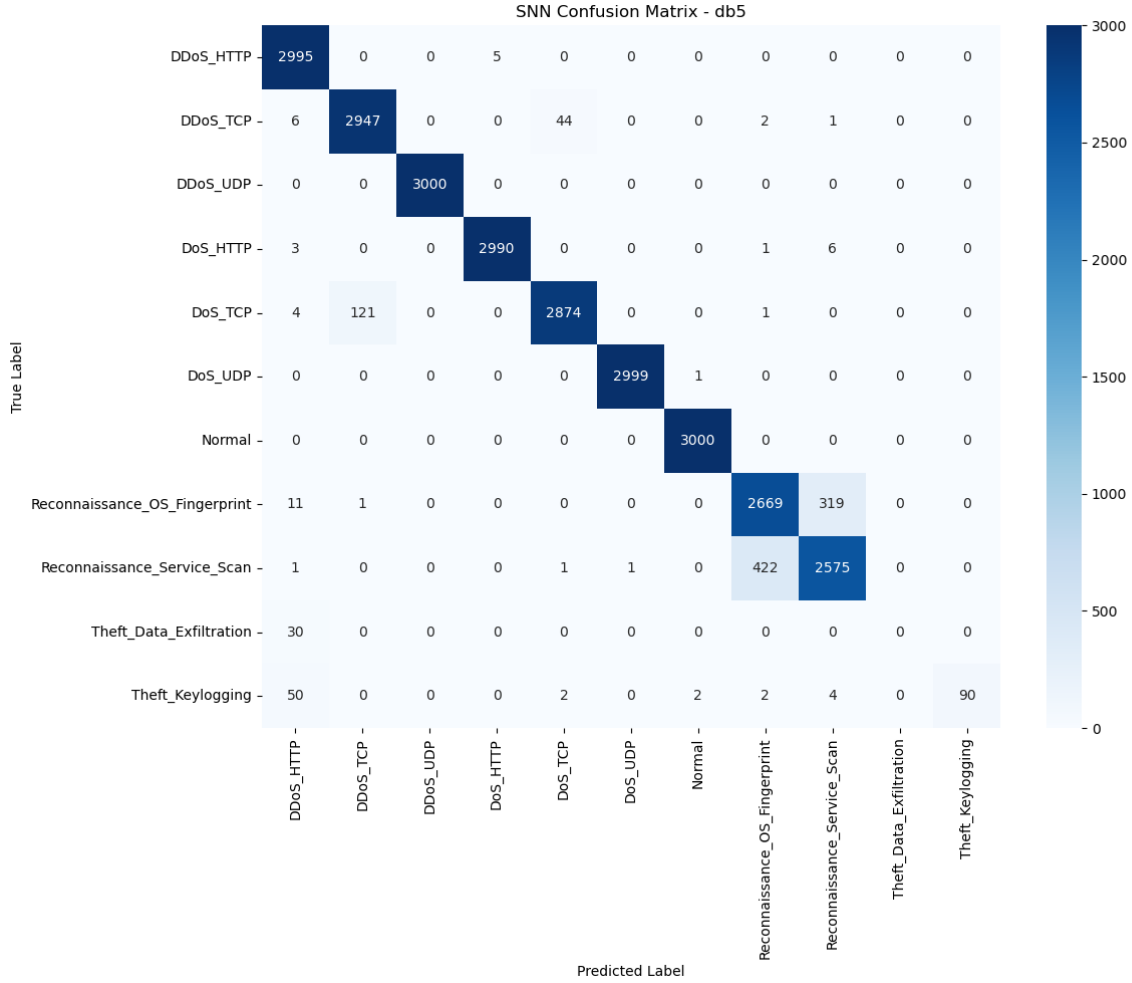


Figure 4.4: Confusion Matrix of SNN model trained on DB5.

DB2 shows significant confusion between normal traffic and major attacks (DDoS_TCP, DDoS_HTTP), with rare benign samples frequently misclassified as attacks. With SMOTE in DB3, attack classes improve, but normal traffic still suffers from false positives due to insufficient diversity. When normal traffic is augmented in DB4, SNN’s discrimination improves dramatically: the “Normal” row aligns with the diagonal and false positives drop sharply. In DB5, both classes achieve high accuracy. The confusion matrix becomes mostly diagonal, indicating robust separation for well-represented classes, though rare classes like keylogging and data exfiltration remain challenging.

Overall, these matrices demonstrate that balanced, realistic data distribution is essential for practical intrusion detection, resulting in models sensitive to real attacks and safe for real-time use, real-world attack evaluation and prediction quality of the SNN model is discussed in Section 4.0.2.

The evaluation shows that advanced ensemble models and the Spiking Neural Network consistently achieve strong results, especially on balanced and realistically augmented datasets. Both normal and minority class samples are classified with significantly improved accuracy. These results confirm the value of data balancing and targeted preprocessing in developing robust and generalizable intrusion detection solutions. The next section focuses on the system’s real-time performance—analyzing detection accuracy, latency, and throughput under live deployment and network conditions.

4.0.2 Real-World Attack Evaluation and Prediction Quality

This section evaluates the model in a real-world attack environment to assess prediction quality. This evaluation highlights the importance of dataset quality and continuous validation against live traffic.

While the models—especially those trained on DB5—demonstrate strong classification metrics on controlled and balanced datasets, their performance differs noticeably when evaluated against real-time attack traffic in live environments. Many DoS and DDoS attacks (including TCP/HTTP variants) are grouped primarily as DDoS_TCP during prediction, with occasional correct classifications of DoS_UDP and DDoS_UDP. Reconnaissance attacks, such as OS fingerprinting and service scans, are detected consistently and accurately. However, more subtle attack categories like data exfiltration and keylogging are often misclassified as reconnaissance or DoS attacks.

This discrepancy likely stems from the Bot-IoT dataset’s nature and collection methodology. The dataset originates from a lab-generated environment with controlled configurations and time windows that differ significantly from real-world conditions. Additionally, the limited number of real samples for some attack types required synthetic oversampling with SMOTE. While SMOTE aids training, it cannot fully capture unseen real-world attack patterns. These factors collectively explain the divergence between real-time prediction accuracy and offline evaluation metrics.

This section highlights the critical importance of dataset quality, representativeness, and continuous validation against live traffic. While the model’s offline performance is promising, ongoing real-world testing and dataset refinement remain essential for successful operational deployment. The next sections provides a comparative analysis of the proposed system against the base paper’s classical machine learning approaches.

4.0.3 Comparative Analysis

This section benchmarks the enhanced models—including the Spiking Neural Network (SNN) and real-time evaluation—against the base paper’s classical machine learning approaches (Decision Tree, Random Forest, Bagging, KNN, LDA, SVM) trained on the Bot-IoT dataset. The base paper lacks neuromorphic modeling and real-time system evaluation, making our framework a significant methodological and practical extension.

The proposed system, incorporating both classical machine learning classifiers and an SNN, significantly extends the capabilities described in the base Bot-IoT study. While the base paper reports high accuracy with traditional classifiers on offline datasets, it does not explore SNNs or address real-time detection challenges.

Classical ensemble models maintain excellent performance (accuracy often exceeding 99%), aligning closely with the base paper’s results. However, the SNN introduces energy-efficient, event-driven learning with competitive accuracy (above 96% on DB5) and promising generalization to real-time scenarios.

Unlike the original study, our work integrates data balancing techniques—SMOTE and synthetic normal data augmentation—which substantially improve minority class recognition and reduce false positives for normal traffic. This particularly benefits SNN performance, where imbalance and limited data previously hindered classification of rare attack types.

Our real-time prediction pipeline, validated on live network traffic, reveals practical constraints absent from the base paper. While the SNN and classical models accurately classify most attacks, real-time conditions expose class confusion—primarily among DoS and DDoS variants due to differences in dataset generation, attack timing, and limited real attack samples.

The comparative analysis underscores the advancement achieved by introducing SNN-based models and real-time deployment, while highlighting areas needing improvement: enhanced dataset realism and further fine-tuning for operational readiness. The evaluation confirms the feasibility of neuromorphic and hybrid approaches in modern IoT intrusion detection beyond traditional batch learning.

4.0.4 Limitations and Future Work

This section outlines key challenges and pathways for system enhancement. The primary limitation stems from the Bot-IoT dataset’s lab-generated nature, which differs significantly from real-world network traffic in configuration, timing, and attack diversity. While SMOTE balancing is necessary for minority classes, it may not fully capture the complexity of unseen attacks, limiting generalization in live environments. Real-time prediction errors—particularly misclassifications among similar attack classes—highlight the need for continual model adaptation and dataset refinement.

Future work should explore reinforcement learning (RL) approaches that continuously learn and adapt to specific network traffic patterns, improving detection accuracy over time for targeted edge device environments. However, RL models require dedicated training on custom datasets that reflect the deployment environment, necessitating initial setup and manual data generation. The Argus tool’s real-time dataset generation capability should be leveraged in future implementations to create tailored, high-fidelity training data streams critical for boosting live detection accuracy. Additional promising directions include attack pattern drift detection, automated model retraining pipelines, and hybrid IDS-IPS integration to strengthen intrusion detection in dynamic IoT networks.

This section conclusively demonstrates the effectiveness of integrating classical machine learning models and a Spiking Neural Network for intrusion detection in IoT environments. While high accuracy and robust classification were achieved across multiple datasets and deployment scenarios, persistent challenges remain in minority class detection and real-time applicability due to dataset limitations. These findings underscore both the promise and the complexity of building practical, adaptive cybersecurity systems.

Chapter 5

Conclusion

This work demonstrates that a Spiking Neural Network (SNN) model trained on the balanced and augmented DB5 version of the Bot-IoT dataset forms the core of an effective IoT intrusion detection framework. The hybrid framework combines classical ensemble classifiers with SNN to deliver robust detection accuracy above 96% while addressing minority class prediction and real-time detection challenges. The system integrates the full pipeline—from network packet capture using Argus to real-time flow analysis and alert visualization—deployed in a containerized environment simulating edge devices. While the dataset’s synthetic nature and live traffic variations present some limitations, the overall results validate the framework’s potential for adaptive, efficient, and scalable IoT security. Future enhancements will explore reinforcement learning-based adaptive models trained on custom locally generated datasets to improve responsiveness and specificity for deployed environments. This work lays a strong foundation for further research and practical deployment of neuromorphic-assisted intrusion detection in IoT networks.

References

- [1] Nickolaos Koroniotis et al. “Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iotdataset”. In: *Future Generation Computer Systems* 100 (2019), pp. 779–796.
- [2] Shema Alosaimi and Saad M. Almutairi. “An Intrusion Detection System Using BoT-IoT”. In: *Applied Sciences* 13.9 (2023), p. 5427.
- [3] Zhen Wang et al. “An efficient intrusion detection model based on spiking neural networks”. In: *Scientific Reports* 14.1 (2024), p. 7054.
- [4] Cynthia Nicho, Prateek Mathur, and Shahmohamad Salim. “Detection of IoT Botnet Cyber Attacks using Machine Learning Algorithms”. In: *Informatica* 47.6 (2023).
- [5] Md Mahbubur Rahman, Shaharia Al Shakil, and Mizanur Rahman Mustakim. “A survey on intrusion detection system in IoT networks”. In: *Journal of Network and Computer Applications* 233 (2025), p. 103986.
- [6] Pranjal Sharma et al. “Leveraging AI for Intrusion Detection in IoT Ecosystems”. In: *IEEE Transactions on Industrial Informatics* (2025).
- [7] Marina S. Amine, Fayza A. Nada, and Khalid M. Hosny. “Improved model for intrusion detection in the Internet of Things”. In: *Scientific Reports* 15.1 (2025), p. 21547.
- [8] Munder Qaddos, Raneem S. Al-Qassas, and Adnan Akhunzada. “A novel intrusion detection framework for optimizing IoT security using hybrid CNN-GRU models”. In: *Scientific Reports* 14.1 (2024), p. 21789.
- [9] S. Manivannan et al. “Recent endeavors in machine learning-powered intrusion detection systems for IoT”. In: *Computer Networks* 241 (2024), p. 109995.
- [10] Omar A. Alghanam et al. “An improved PIO feature selection algorithm for IoT network intrusion detection system based on ensemble learning”. In: *Expert Systems with Applications* 213 (2023), p. 119005.

-
- [11] Ankit Thakkar and Ritika Lohiya. “Attack classification of imbalanced intrusion data for IoT network using ensemble learning-based deep neural network”. In: *IEEE Internet of Things Journal* 10.15 (2023), pp. 13589–13600.
 - [12] Maryam Douiba et al. “An improved intrusion detection system using gradient boosting and decision tree algorithms for IoT security”. In: *The Journal of Supercomputing* 79.3 (2023), pp. 3392–3411.
 - [13] Salifou Yarga and David H. Wood. “Direct training of high-performance deep spiking neural networks”. In: *Frontiers in Neuroscience* 18 (2024).
 - [14] PM Fazlur Rahman et al. “Exploring spiking neural networks: A comprehensive analysis of models, performance, and limitations”. In: *IEEE Access* 11 (2023), pp. 63647–63665.
 - [15] Fawaz A. Alotaibi and Sanjay Mishra. “Cyber Security Intrusion Detection and Bot Data Collection using Deep Learning in the IoT”. In: *International Journal of Advanced Computer Science and Applications* 15.1 (2024).